

IMDb Movie Review Sentiment Analysis using NLP and Machine Learning

Project Overview

This project focuses on analyzing movie reviews from the IMDb dataset and predicting whether a review is **Positive** or **Negative** using **Natural Language Processing (NLP)** and **Machine Learning** techniques.

The goal is to clean textual data, perform exploratory data analysis (EDA), transform text into numerical features, and build a classification model capable of predicting sentiment.

Dataset Information

Dataset: IMDb Movie Reviews

Target Variable:

- Positive Review → 1
- Negative Review → 0

Features:

- Review (Text)
 - Sentiment (Target)
-

◆ Step 1: Import Required Libraries

The first step is importing Python libraries for:

Data Manipulation

- Pandas
- NumPy

Visualization

- Matplotlib
- Seaborn

NLP

- NLTK
- WordCloud

Machine Learning

- Scikit-Learn

Purpose:

These libraries help in data cleaning, visualization, text preprocessing, and model building.

◆ Step 2: Load Dataset

```
data = pd.read_csv("imdb.csv")
```

Initial Inspection

```
data.head()  
data.shape  
data.info()
```

Purpose:

- Understand dataset structure
 - Check column names
 - Identify data types
-

◆ Step 3: Data Cleaning

Missing Values Check

```
data.isnull().sum()
```

Duplicate Values Check

```
data.duplicated().sum()
```

Remove Duplicate Records

```
data.drop_duplicates(inplace=True)
```

Purpose:

- Improve data quality
 - Remove redundant observations
 - Increase model reliability
-

◆ Step 4: Target Variable Analysis

Sentiment Distribution

```
data['sentiment'].value_counts()
```

Visualization:

```
sns.countplot(x='sentiment', data=data)
```



Insights:

- Number of positive reviews
 - Number of negative reviews
 - Dataset balance
-

◆ Step 5: Convert Target Variable

Convert categorical labels into numeric values.

```
data['sentiment'] = data['sentiment'].map({  
    'positive':1,  
    'negative':0  
})
```

Result:

Sentiment Value

Positive 1

Negative 0

Purpose:

Machine learning algorithms require numerical values.

◆ Step 6: Text Preprocessing

Text preprocessing is the most important step in NLP.

6.1 Convert Text to Lowercase

```
data['review'] = data['review'].str.lower()
```

Example:

Before:

"I LOVE THIS MOVIE"

After:

"i love this movie"

6.2 Remove HTML Tags

Example:

Before:

```
<p>Great Movie!</p>
```

After:

```
Great Movie!
```

Purpose:

Remove unwanted website formatting.

6.3 Remove Special Characters

Before:

```
Amazing!!! Movie@@@
```

After:

```
Amazing Movie
```

Purpose:

Keep only meaningful words.

6.4 Remove Stopwords

Examples:

- is
- the
- and
- of
- a

Purpose:

Remove common words that add little meaning.

6.5 Lemmatization

Examples:

Original Lemma

Running Run

Dogs Dog

Better Good

Purpose:

Reduce words to their root form.

◆ Step 7: Exploratory Data Analysis (EDA)

EDA helps understand the review patterns.

Dataset Overview

```
print(data.shape)
```

Review Length Analysis

Create a new feature:

```
data['review_length'] = data['review'].apply(len)
```

Statistics:

```
data['review_length'].describe()
```

Review Length Distribution

```
plt.figure(figsize=(8,5))  
sns.histplot(data['review_length'])
```

Insight

- Short reviews
- Medium reviews
- Long reviews

Understand user behavior.

◆ Step 8: Most Frequent Words Analysis

Using Counter:

```
from collections import Counter
```

Find top occurring words.

Visualization:

Bar Chart of Top 20 Words

 Insight:

Most commonly used words in reviews.

◆ Step 9: Word Cloud Visualization

Generate Word Clouds for:

Positive Reviews

```
WordCloud()
```

Shows words frequently used in positive reviews.

Examples:

- Great
 - Excellent
 - Amazing
 - Best
-

Negative Reviews

`WordCloud()`

Shows words frequently used in negative reviews.

Examples:

- Bad
 - Boring
 - Worst
 - Waste
-

◆ Step 10: Feature Engineering

Machine learning models cannot understand text directly.

Convert text into numbers using TF-IDF.

TF-IDF Vectorization

`TfidfVectorizer()`

Purpose:

Transform text into numerical vectors.

Benefits:

- Captures word importance
- Reduces effect of common words
- Improves prediction performance

◆ Step 11: Train-Test Split

```
train_test_split()
```

Split:

- Training Data = 80%
- Testing Data = 20%

Purpose:

Evaluate model performance on unseen reviews.

◆ Step 12: Machine Learning Model

Logistic Regression

```
LogisticRegression(max_iter=1000)
```

Why Logistic Regression?

- Simple
 - Fast
 - Effective for text classification
-

◆ Step 13: Model Training

```
lr.fit(X_train, y_train)
```

The model learns patterns from training reviews.

◆ Step 14: Prediction

```
lr_pred = lr.predict(X_test)
```

Purpose:

Predict sentiment for unseen reviews.

◆ Step 15: Model Evaluation

Accuracy Score

```
accuracy_score(y_test, lr_pred)
```

Measures overall correctness.

Classification Report

```
classification_report()
```

Metrics:

- Precision
 - Recall
 - F1 Score
-

Confusion Matrix

```
confusion_matrix()
```

Visualization:

```
sns.heatmap()
```

Interpretation:

Actual	Predicted
TP	Correct Positive
TN	Correct Negative
FP	Wrong Positive
FN	Wrong Negative



Visualizations Included

- ✓ Sentiment Distribution Count Plot

- ✓ Review Length Histogram
 - ✓ Review Length Box Plot
 - ✓ Top Frequent Words Bar Chart
 - ✓ Positive Review Word Cloud
 - ✓ Negative Review Word Cloud
 - ✓ Confusion Matrix Heatmap
-



Key Findings

- Dataset contains positive and negative movie reviews.
 - Text preprocessing significantly improves data quality.
 - TF-IDF successfully converts text into numerical features.
 - Logistic Regression performs well for sentiment classification.
 - Word Clouds reveal important sentiment-related keywords.
 - Confusion Matrix helps evaluate classification performance.
-



Technologies Used

- Python
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - NLTK
 - WordCloud
 - Scikit-Learn
-



Conclusion

This project demonstrates a complete NLP pipeline for sentiment analysis. Starting from raw textual reviews, the project performs data cleaning, text preprocessing, exploratory data analysis, feature engineering, machine learning model training, and performance evaluation.

The Logistic Regression model effectively classifies movie reviews into positive and negative sentiments, making it a strong baseline model for sentiment analysis tasks.

IMDb Movie Review Sentiment Analysis using NLP and Machine Learning

Project Overview

This project focuses on analyzing movie reviews from the IMDb dataset and predicting whether a review is **Positive** or **Negative** using **Natural Language Processing (NLP)** and **Machine Learning** techniques.

The goal is to clean textual data, perform exploratory data analysis (EDA), transform text into numerical features, and build a classification model capable of predicting sentiment.

Dataset Information

Dataset: IMDb Movie Reviews

Target Variable:

- Positive Review $\rightarrow 1$
- Negative Review $\rightarrow 0$

Features:

- Review (Text)
 - Sentiment (Target)
-

◆ Step 1: Import Required Libraries

The first step is importing Python libraries for:

Data Manipulation

- Pandas
- NumPy

Visualization

- Matplotlib
- Seaborn

NLP

- NLTK
- WordCloud

Machine Learning

- Scikit-Learn

Purpose:

These libraries help in data cleaning, visualization, text preprocessing, and model building.

◆ Step 2: Load Dataset

```
data = pd.read_csv("imdb.csv")
```

Initial Inspection

```
data.head()  
data.shape  
data.info()
```

Purpose:

- Understand dataset structure
 - Check column names
 - Identify data types
-

◆ Step 3: Data Cleaning

Missing Values Check

```
data.isnull().sum()
```

Duplicate Values Check

```
data.duplicated().sum()
```

Remove Duplicate Records

```
data.drop_duplicates(inplace=True)
```

Purpose:

- Improve data quality
 - Remove redundant observations
 - Increase model reliability
-

◆ Step 4: Target Variable Analysis

Sentiment Distribution

```
data['sentiment'].value_counts()
```

Visualization:

```
sns.countplot(x='sentiment', data=data)
```

 Insights:

- Number of positive reviews
 - Number of negative reviews
 - Dataset balance
-

◆ Step 5: Convert Target Variable

Convert categorical labels into numeric values.

```
data['sentiment'] = data['sentiment'].map({  
    'positive':1,  
    'negative':0  
})
```

Result:

Sentiment Value

Positive 1

Negative 0

Purpose:

Machine learning algorithms require numerical values.

◆ Step 6: Text Preprocessing

Text preprocessing is the most important step in NLP.

6.1 Convert Text to Lowercase

```
data['review'] = data['review'].str.lower()
```

Example:

Before:

"I LOVE THIS MOVIE"

After:

"i love this movie"

6.2 Remove HTML Tags

Example:

Before:

```
<p>Great Movie!</p>
```

After:

```
Great Movie!
```

Purpose:

Remove unwanted website formatting.

6.3 Remove Special Characters

Before:

```
Amazing!!! Movie@@@
```

After:

```
Amazing Movie
```

Purpose:
Keep only meaningful words.

6.4 Remove Stopwords

Examples:

- is
- the
- and
- of
- a

Purpose:
Remove common words that add little meaning.

6.5 Lemmatization

Examples:

Original Lemma

Running Run

Dogs Dog

Better Good

Purpose:
Reduce words to their root form.

◆ Step 7: Exploratory Data Analysis (EDA)

EDA helps understand the review patterns.

Dataset Overview

```
print(data.shape)
```

Review Length Analysis

Create a new feature:

```
data['review_length'] = data['review'].apply(len)
```

Statistics:

```
data['review_length'].describe()
```

Review Length Distribution

```
plt.figure(figsize=(8,5))  
sns.histplot(data['review_length'])
```

Insight

- Short reviews
- Medium reviews
- Long reviews

Understand user behavior.

◆ Step 8: Most Frequent Words Analysis

Using Counter:

```
from collections import Counter
```

Find top occurring words.

Visualization:

Bar Chart of Top 20 Words

 Insight:

Most commonly used words in reviews.

◆ Step 9: Word Cloud Visualization

Generate Word Clouds for:

Positive Reviews

`WordCloud()`

Shows words frequently used in positive reviews.

Examples:

- Great
 - Excellent
 - Amazing
 - Best
-

Negative Reviews

`WordCloud()`

Shows words frequently used in negative reviews.

Examples:

- Bad
 - Boring
 - Worst
 - Waste
-

◆ Step 10: Feature Engineering

Machine learning models cannot understand text directly.

Convert text into numbers using TF-IDF.

TF-IDF Vectorization

`TfidfVectorizer()`

Purpose:

Transform text into numerical vectors.

Benefits:

- Captures word importance
 - Reduces effect of common words
 - Improves prediction performance
-

◆ Step 11: Train-Test Split

```
train_test_split()
```

Split:

- Training Data = 80%
- Testing Data = 20%

Purpose:

Evaluate model performance on unseen reviews.

◆ Step 12: Machine Learning Model

Logistic Regression

```
LogisticRegression(max_iter=1000)
```

Why Logistic Regression?

- Simple
 - Fast
 - Effective for text classification
-

◆ Step 13: Model Training

```
lr.fit(X_train, y_train)
```

The model learns patterns from training reviews.

◆ Step 14: Prediction

```
lr_pred = lr.predict(X_test)
```

Purpose:
Predict sentiment for unseen reviews.

◆ Step 15: Model Evaluation

Accuracy Score

```
accuracy_score(y_test, lr_pred)
```

Measures overall correctness.

Classification Report

```
classification_report()
```

Metrics:

- Precision
 - Recall
 - F1 Score
-

Confusion Matrix

```
confusion_matrix()
```

Visualization:

```
sns.heatmap()
```

Interpretation:

Actual	Predicted
TP	Correct Positive
TN	Correct Negative
FP	Wrong Positive
FN	Wrong Negative



Visualizations Included

- ✓ Sentiment Distribution Count Plot
 - ✓ Review Length Histogram
 - ✓ Review Length Box Plot
 - ✓ Top Frequent Words Bar Chart
 - ✓ Positive Review Word Cloud
 - ✓ Negative Review Word Cloud
 - ✓ Confusion Matrix Heatmap
-



Key Findings

- Dataset contains positive and negative movie reviews.
 - Text preprocessing significantly improves data quality.
 - TF-IDF successfully converts text into numerical features.
 - Logistic Regression performs well for sentiment classification.
 - Word Clouds reveal important sentiment-related keywords.
 - Confusion Matrix helps evaluate classification performance.
-



Technologies Used

- Python
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - NLTK
 - WordCloud
 - Scikit-Learn
-



Conclusion

This project demonstrates a complete NLP pipeline for sentiment analysis. Starting from raw textual reviews, the project performs data cleaning, text preprocessing, exploratory data analysis, feature engineering, machine learning model training, and performance evaluation.

The Logistic Regression model effectively classifies movie reviews into positive and negative sentiments, making it a strong baseline model for sentiment analysis tasks.